



UNIVERSITÀ DEGLI STUDI DI CATANIA
DIPARTIMENTO DI MATEMATICA E INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INFORMATICA

Petronio Petronio Davide

Computer Security

APPUNTI LEZIONI

Prof. G. Bella

Anno Accademico 2025 - 2026

Indice

1	Symmetric Needham-Schroeder e Kerberos	5
1.1	Kerberos	7
1.1.1	L'Integrazione di Kerberos con le Smart Card	11
2	Smart Card	14
2.1	Standard ISO 7816 e Attacchi	16
2.1.1	Tipologie di Attacco	16
3	Crittografia Visuale	19
4	Non-Ripudio	23
4.1	Problema della equa Compravendita	24
4.1.1	Mezzo affidabile, Agenti Onesti	24
4.1.2	Mezzo affidabile, Agenti disonesti	25
4.1.3	Mezzo inaffidabile, Agenti onesti	25
4.1.4	Mezzo inaffidabile, Agenti disonesti	26
4.1.5	Protocollo Zhou-Gollman	26
4.1.6	Analisi del rilascio posticipato	28
4.2	Non ripudio in e-mail	28
4.2.1	Equo recapito	28
4.2.2	Protocollo Abadi et al.	29
4.2.2.1	Risoluzione delle controversie	32
4.3	Non ripudio in Delega	32
4.3.1	Protocollo Crispo	33
4.3.1.1	Risoluzione delle controversie	34
4.3.2	Semplificazione del Protocollo	35
5	Analisi Formale	36
5.1	Model Checking	37
5.2	Theorem Proving	38
5.2.1	Object Level vs Meta Level	40

<i>INDICE</i>	2
5.3 Isabelle	40
5.3.1 I Tipi Fondamentali (Datatypes)	41
5.3.2 Operatori di Conoscenza	41
5.3.3 Definizione Induttiva del Protocollo	42

Introduzione

Un punto di partenza fondamentale per lo studio della materia è la distinzione, nonché il profondo legame, tra il concetto di **Privacy** e quello di **Data Protection** (Protezione dei dati), sanciti nella Carta dei diritti fondamentali dell'Unione Europea:

- **Articolo 7 - Rispetto della vita privata e della vita familiare:** Sancisce la nascita e il riconoscimento formale della privacy. La privacy viene qui definita come un vero e proprio **diritto fondamentale del cittadino**, ovvero il diritto a non subire ingerenze non giustificate nella propria sfera privata.
- **Articolo 8 - Protezione dei dati di carattere personale:** Introduce il concetto di *Data Protection*. La protezione dei dati non è semplicemente una regola tecnica, ma è il **modo pratico per realizzare e tutelare la privacy**. Affinché il trattamento dei dati (*data processing*) sia lecito, è necessario che vi sia il **consenso inequivocabile e chiaro** della persona interessata. Inoltre, questo articolo garantisce all'individuo il diritto inalienabile di poter accedere e visionare i propri dati raccolti da terzi.

Questa distinzione ci porta a chiarire un ulteriore dualismo, spesso fonte di confusione: quello tra *Security* (Sicurezza) e *Privacy* (Riservatezza). Sebbene siano spesso confusi, affrontano problemi diversi. Mentre la sicurezza si concentra sulle misure tecniche atte a proteggere i dati da accessi non autorizzati e attacchi informatici, la privacy riguarda il diritto dell'utente di controllare l'uso che viene fatto delle proprie informazioni. Un sistema può essere tecnicamente molto sicuro, ma violare comunque la privacy degli utenti se raccoglie e utilizza dati senza un consenso trasparente.

Proprio per restituire all'utente il controllo sulle proprie informazioni, le normative moderne conferiscono ai cittadini strumenti potenti. Tra i diritti dell'interessato spiccano in particolare:

- **Diritto alla cancellazione (o Diritto all'oblio):** Consente all'utente di richiedere e ottenere l'eliminazione definitiva dei propri dati personali dai sistemi di un'organizzazione.
- **Diritto alla portabilità:** La possibilità per l'utente di ricevere i propri dati in un formato strutturato e di trasferirli facilmente da un fornitore di servizi a un altro (es. da un social network all'altro).

La corretta gestione di queste complesse dinamiche all'interno delle aziende richiede l'intervento di figure professionali con responsabilità ben distinte e complementari:

- **CTO (Chief Technology Officer):** Ha la responsabilità dell'infrastruttura tecnologica e delle scelte di sviluppo IT dell'azienda.
- **CISO (Chief Information Security Officer):** È il responsabile della sicurezza informatica pura. Si assicura che le reti e i sistemi aziendali siano protetti contro intrusioni, malware e data breach.
- **DPO (Data Protection Officer):** È il responsabile della protezione dei dati. Rispetto al CISO, il DPO affronta **problematiche molto più ampie e trasversali**. Non si limita a verificare la solidità dei firewall, ma deve garantire che l'intera organizzazione rispetti le normative (come il GDPR). È la figura che si occupa attivamente di gestire i diritti degli utenti, come ad esempio supervisionare le pratiche legali e tecniche per garantire l'**eliminazione** dei dati o consentirne il corretto **accesso** all'utente che ne fa richiesta.

Capitolo 1

Symmetric Needham-Schroeder e Kerberos

Passiamo ora ad analizzare il protocollo **Symmetric Needham-Schroeder**, che pone le basi teoriche e strutturali per quello che sarà poi Kerberos. Ci troviamo in un contesto di crittografia simmetrica, in cui ogni entità condivide una chiave segreta a "lungo termine" con un Trusted Third Party (TTP). Lo scopo del protocollo è permettere al TTP di generare e distribuire in modo sicuro una chiave di sessione (K_{ab}) affinché due nodi, A e B , possano comunicare.

Il protocollo si sviluppa nei seguenti cinque passaggi:

1. $A \rightarrow TTP : A, B, N_a$
2. $TTP \rightarrow A : \{N_a, B, K_{ab}, \{K_{ab}, A\}_{K_b}\}_{K_a}$
3. $A \rightarrow B : \{K_{ab}, A\}_{K_b}$
4. $B \rightarrow A : \{N_b\}_{K_{ab}}$
5. $A \rightarrow B : \{N_b - 1\}_{K_{ab}}$

Analizzando la struttura di questo scambio di messaggi, emergono alcuni dettagli e criticità importanti:

- **Freshness e prevenzione dei Replay Attack:** I passaggi 4 e 5 servono a B per ottenere garanzia di *freshness* sulla chiave di sessione appena ricevuta. L'operazione di decurtazione (il -1 applicato alla nonce N_b nel passaggio 5) è fondamentale per differenziare i due messaggi; se fossero identici, un attaccante potrebbe semplicemente intercettare il messaggio 4 e rispedirlo identico al mittente.
- **Mancanza di Explicitness:** Il protocollo pecca nel principio di esplicitezza. Al passaggio 4, A riceve un messaggio crittografato ma non ha

la prova esplicita di chi glielo stia mandando. Similmente, quando B riceve la nonce indietro al passaggio 5, *presume* che sia stata elaborata e inviata da A , ma il messaggio in sé non ne contiene l'identità.

Queste vulnerabilità aprono le porte a un attacco critico se consideriamo un modello di minaccia più avanzato. Introduciamo quindi l'attaccante **Dolev-Yao Plus**, un'estensione del classico modello Dolev-Yao in cui l'attaccante non solo può intercettare, bloccare e falsificare i messaggi, ma è anche riuscito a impossessarsi di vecchie chiavi di sessione ormai compromesse.

Sfruttando una vecchia chiave di sessione (K_{ab}) e il relativo ticket crittografato ($\{K_{ab}, A\}_{K_b}$), l'attaccante C può perpetrare un **Replay Attack** (noto anche come attacco di Denning-Sacco) riutilizzando il passaggio 3 e impersonando A :

3. $C \rightarrow B : \{K_{ab}, A\}_{K_b}$ (Ticket vecchio intercettato precedentemente)
4. $B \rightarrow C : \{N'_b\}_{K_{ab}}$ (B risponde all'attaccante credendo sia A)
5. $C \rightarrow B : \{N'_b - 1\}_{K_{ab}}$ (C , conoscendo la vecchia chiave, calcola la risposta)

Il cuore del problema risiede nel messaggio 5. Questo messaggio dimostra a B soltanto che "qualcuno che conosce la chiave di sessione ha letto e processato la nonce". Non si tratta di un problema di autenticazione in senso stretto (che si risolverebbe aggiungendo i nomi all'interno dei messaggi crittografati per garantire l'esplicitatezza), bensì di un problema di **freshness dell'autenticazione**: B non ha alcuna garanzia che la richiesta di sessione sia *attualmente* voluta da A .

Per mitigare questa falla strutturale, la soluzione consiste nell'includere una nonce generata da B fin dall'inizio nella produzione della chiave di sessione.

1. $A \rightarrow B : A$
2. $B \rightarrow A : \{A, N'_b\}_{K_b}$
3. $A \rightarrow TTP : A, B, N_a, \{A, N'_b\}_{K_b}$
4. $S \rightarrow A : \{N_a, K_{ab}, B, \{K_{ab}, A, N'_b\}_{K_b}\}$

Per risolvere il problema della freschezza dell'autenticazione emerso con il protocollo Needham-Schroeder, si è resa necessaria un'evoluzione. Vogliamo un protocollo sicuro che risolva i limiti del precedente: per farlo, si passò dall'uso delle nonce all'utilizzo dei **timestamp**. Sebbene all'epoca l'uso dei timestamp non fosse considerato pratico principalmente per le difficoltà legate alla sincronizzazione degli orologi tra macchine diverse, il protocollo **Denning-Sacco** fu il primo a implementarli con successo. Questo filone di

ricerca gettò le basi per il **Progetto Athena** del MIT, dal quale è nato il protocollo che oggi conosciamo come **Kerberos**.

Progettare un sistema di sicurezza per una rete locale (LAN) impone di trovare il giusto compromesso tra **Autenticazione** e **Usabilità**. L'obiettivo è permettere agli utenti di accedere in maniera sicura ai vari servizi offerti dalla nostra LAN.

Tuttavia, emerge subito un problema di usabilità: se ogni accesso a un nuovo servizio di rete causasse una nuova procedura di autenticazione, l'esperienza utente sarebbe inefficiente poiché dovrebbe reinserire di volta in volta le sue credenziali. La soluzione a questo problema è l'adozione di un meccanismo di **Single Sign-On (SSO)**. Analizziamo i concetti chiave del Single Sign-On:

- **Definizione:** Il Single Sign-On consiste nell'usare un'unica credenziale di autenticazione per accedere a tutti i servizi. In pratica, l'utente si autentica una sola volta all'inizio della sessione ricevendo un "token" o "ticket" che gli garantirà automaticamente l'accesso trasparente a tutti i servizi distribuiti successivi.
- **Il problema della robustezza:** Di base, l'SSO è una soluzione indubbiamente comoda, ma intrinsecamente poco robusta se implementata in modo ingenuo, poiché si riduce ad avere un'unica password per tutto. Se un attaccante compromettesse quell'unica credenziale, avrebbe accesso all'intero ecosistema.

Kerberos è un protocollo reale che si occupa di risolvere questo esatto problema implementando il Single Sign-On proteggendo le credenziali attraverso l'uso di ticket crittografati a tempo che fanno uso di **timestamp**.

1.1 Kerberos

Prima di parlare di Kerberos, mostriamo una prima versione detta **BAN Kerberos** che ha uno schema molto simile a quello che vedremo successivamente.

$$A \rightarrow TTP : A \quad (1.1)$$

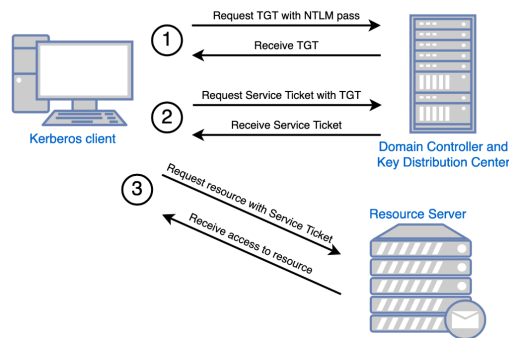
$$TTP \rightarrow A : \{K_{ab}, B, T_k, \{K_{ab}, A, T_k\}_{K_b}\}_{K_a} \quad (1.2)$$

$$A \rightarrow B : \{K_{ab}, A, T_k\}_{K_b} \quad (1.3)$$

Come già detto, Kerberos si pone come obiettivo quello di realizzare un sistema a Single Sign-On che permette una **singola autenticazione** al sistema ma che faccia un **controllo delle autorizzazioni** quando l'utente vuole accedere a specifici punti del sistema.

CAPITOLO 1. SYMMETRIC NEEDHAM-SCHROEDER E KERBEROS

Questo, inoltre, fa uso di timestamp che si differenziano dalle Nonce principalmente per il fatto che la nonce dà garanzia a chi la mette della freschezza di colui che la rimanda indietro poiché prima di quel momento la nonce non era stata creata. Il timestamp, invece, non dà alcuna garanzia a colui che la riceve che questa sia veritiera poiché colui che la manda potrebbe essere malevolo e mandare un timestamp più nuovo. Se, da una parte, abbiamo delle Nonce che ci indicano chiavi "vecchie" che sono insicure, dall'altra abbiamo i Timestamp che indicano chiavi "scadute" che sono insicure. La differenza sta nel fatto che una chiave "vecchia" non ci sta a indicare un qualche riferimento temporale in termini di tempo mentre, invece, quando la scadenza dettata dal timestamp ci fissa un limite preciso nel tempo. Dunque, adesso che utilizzeremo i timestamp, ci fideremo solo di quelli generati da un entità fidata. Gli obiettivi di Kerberos sono segretezza, autenticazione ad accesso singolo e temporalità che ci permette di prevenire replay. La struttura base di Kerberos è la seguente:



Come possiamo notare, abbiamo principalmente tre fasi:

1. **Autenticazione:** Il client si autentica con l'autenticazione server (AS). Questa fase fornirà al client l'**AuthKey** che sarebbe la chiave di sessione per comunicare con il TGS e l'**AuthTicket** che sarebbe un ticket che A invierà al TGS per comunicargli la chiave di sessione. Questo Ticket sarà cifrato con la chiave privata del TGS così che il client che la richiede non possa modificarlo.
2. **Autorizzazione:** Il client chiede al Ticket Granting Server (TGS) un ticket per uno specifico servizio. Nello stesso identico modo di prima, il TGS fornisce al client la **ServKey** e il **ServTicket**.

L'unica operazione obbligatoria è la prima mentre le operazioni due e tre vengono effettuate solo se richieste dal client poiché non è detto che questo

richieda dei servizi. Inoltre, con l'aggiunta dei Timestamp diventa anche necessario parlare di tempo di validità delle sessioni e delle chiavi. Infatti, con l'aggiunta dei timestamp nasce il concetto di "accetto questa operazione solo se il timestamp ha un valore minore rispetto a quello che sono disposto ad accettare". Dunque, per realizzare il Single Sign-On, avremo le credenziali di autenticazione hanno una valenza di circa 8 ore che sarebbero le ore di una tipica giornata lavorativa mentre, invece, le credenziali per i servizi hanno una valenza di al massimo qualche minuto poiché abbiamo la necessità che queste siano fresche per prevenire grosse compromissioni del sistema. Lo schema Kerberos funziona come segue:

Autenticazione:

1. $A \rightarrow AS : A, TGS, T_1$
2. $AS \rightarrow A : \{authK, TGS, T_a, \{A, TGS, authK, T_a\}_{K_{tgs}}\}_{K_a}$

Autorizzazione:

3. $A \rightarrow TGS : \{A, TGS, authK, T_a\}_{K_{tgs}}, \{A, T_2\}_{authK}, B$
4. $TGS \rightarrow A : \{servK, B, T_s, \{A, B, servK, T_s\}_{K_b}\}_{authK}$

Servizio:

5. $A \rightarrow B : \{A, B, servK, T_s\}_{K_b}, \{A, T_3\}_{servK}$
6. $B \rightarrow A : \{T_3 + 1\}_{servK}$

L'AuthTicket e il ServTicket che dicevamo prima sono rispettivamente $\{A, TGS, authK, T_a\}_{K_{tgs}}$ e $\{A, B, servK, T_s\}_{K_b}$. Da qui possiamo fare alcune considerazioni:

- **Perché T1, T2 e T3:** se, come abbiamo detto, ci fidiamo solo di timestamp creati da entità fidate, a quale scopo inserire i timestamp nei messaggi di A? Infatti, possiamo notare che queste non sono effettivamente fidate, infatti abbiamo sempre timestamp inviati o dall'AS o dal TGS. Nel caso di T_1 , questo serve ad A solo per marcare la sua richiesta e non è nemmeno cifrato, dunque è quasi una formalità. Nel caso di T_2, T_3 non ci si fida realmente dei timestamp, ci si fida del fatto che quelli inviati da A sono legati dalla cifratura che è, a sua volta, legata al timestamp inviato dall'entità fidata.
- **Concatenazione del ticket e dell'autenticatore in 3 e 5:** Notiamo che i due messaggi sono concatenati e, sappiamo da internet security, non è possibile autenticare la concatenazione di due messaggi separati. Tuttavia, anche se un attaccante provasse a riciclare la seconda parte del messaggio, i timestamp fungerebbero da segnale d'allarme e chiunque se ne accorgerebbe.

- **Perché B al messaggio 3 non è cifrato:** Il motivo per il quale "B" al messaggio 3 è inviato senza l'uso di alcun tipo di cifratura è dettato dal fatto che in realtà questa informazione non è necessario nascondere dato che anche se l'attaccante lo sostituisse, al messaggio 4 arriverebbe ad A il messaggio con la *servK* associata ad un altro servizio. Inoltre, anche nascondendo questa informazione, poi al messaggio 5 si scoprirebbe a quale servizio sta facendo accesso.
- **Il messaggio 6:** Come nel caso di Needham Schroeder simmetrico, anche in questo caso viene utilizzato quel $+1$ per differenziare il messaggio 6 dalla seconda parte del messaggio 5 ed evitare problemi. Proprio per lo stesso motivo, quando A legge $T_3 + 1$ cifrato con la *servK*, questo **dedurrà** di star comunicando con B e questa deduzione può essere sfruttata da Dolev-Yao $+$.
- **Gerarchia delle chiavi:** Come possiamo notare, al passo 4 abbiamo che cifriamo *servK* con *authK* di fatto legando indissolubilmente due chiavi di sessione. Infatti, nel caso in cui un attaccante riuscisse a "rubare" *authK*, questo avrebbe accesso automaticamente anche a *servK*.
- **Scadenza delle chiavi di sessione:** Una volta scaduta la sessione di autenticazione, che cosa accade alle chiavi di sessione dei servizi ancora attive? È naturale pensare queste semplicemente smettano di essere valide, tuttavia non è un dettaglio da trascurare poiché, nel caso in cui DY+ riuscisse a impadronirsi di *authK* scaduta, avrebbe modo di scoprire una *servKey* ancora valida. Dunque, è necessario prendere delle precauzioni in questo senso imponendo che la validità della *servK* sia limitata dalla validità di *authK*.
Quando B si trova davanti quella *servK*, poiché sa che vale che $Ts + Ls \leq Ta + La$ ¹, sa che non può essere scaduta.
- **Cifratura annidata:** Una cosa che possiamo notare al passo 2 e al passo 4 è la doppia cifratura del messaggio esterno e del ticket interno. Questa trovata fa' sì che colui che poi riceve il ticket, riesca ad avere garanzia che il suo interlocutore lo abbia ricevuto pure.² Se non ci fosse questa cifratura annidata ma ci fossero solamente due messaggi e nel messaggio 3 e 5 non ci fosse l'autenticatore, un attaccante potrebbe intercettare il messaggio e inviare il Ticket al TGS/Servizio senza

¹ Ts è il tempo in cui è stata generata la *servK* mentre Ls è il suo lifetime. Allo stesso modo vale per Ta ed La .

²Attacco che si può vedere nel protocollo Otway-Rees

che A lo abbia mai ricevuto. Tuttavia, poiché è presente l'autenticatore, questo annidamento è irrilevante poiché, anche se fossero separati, l'attaccante non potrebbe riuscire a formare un messaggio 3 e 5 ben fatto poiché non avrebbe l'authKey/servKey e, quindi, colui che lo riceve capisce subito che qualcosa non è andato correttamente. Questo annidamento è stato successivamente rimosso con Kerberos V.

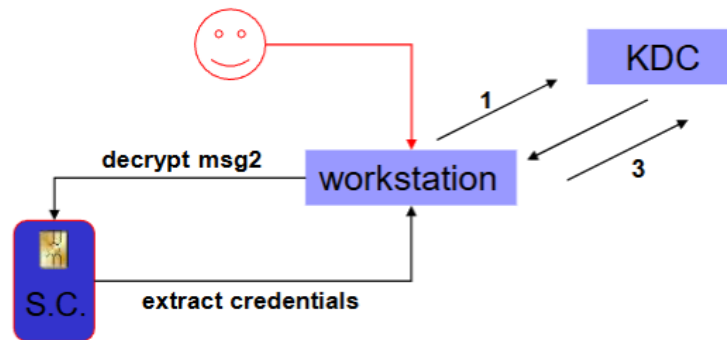
1.1.1 L'Integrazione di Kerberos con le Smart Card

Come già affrontato in altri ambiti (ad esempio nel corso di Internet Security), l'uso delle classiche password solleva numerose criticità: richiedono lunghezze minime, set di caratteri complessi e, soprattutto, pongono il problema fondamentale di come e dove memorizzarle in modo sicuro.

Sebbene si possa pensare di utilizzare una password per derivare la nostra chiave a lungo termine (K_a), nella pratica questa non rappresenta un segreto crittograficamente forte. Anche ricorrendo a tecniche di mitigazione come l'aggiunta di un *salt*, derivare una chiave realmente robusta da una stringa pensata da un essere umano rimane difficile. Ma il problema principale, lato client, è un altro: **dove possiamo memorizzare la chiave K_a in maniera inattaccabile?**

La soluzione a questo problema di archiviazione è l'introduzione della **Smart Card**. Non si tratta di un semplice supporto di memoria passivo, ma di un vero e proprio "mini computer" integrato in un chip. L'utilizzo di questa tecnologia si basa su due principi fondamentali:

- **Inaccessibilità della chiave:** La chiave a lungo termine K_a viene iniettata e salvata all'interno del chip e **non deve mai uscire** da esso. Se la chiave venisse trasferita nella RAM del computer host per essere utilizzata, saremmo punto e a capo con i rischi di compromissione.
- **Elaborazione interna:** Poiché il segreto non può uscire dal chip, è la Smart Card stessa a doversi occupare di eseguire fisicamente le operazioni di cifratura e decifratura richieste dal protocollo Kerberos, restituendo al computer solo il risultato dell'operazione.



Come possiamo osservare dal diagramma, i ruoli sono nettamente separati per garantire la sicurezza:

1. Sarà la **workstation** (il computer fisico) a generare e inviare il primo messaggio in chiaro (msg 1) verso il KDC (Key Distribution Center), dichiarando la propria identità.
2. Il KDC risponde con un messaggio crittografato (msg 2) che contiene, tra le altre cose, la chiave di sessione.
3. A questo punto, la workstation inoltra il messaggio crittografato alla **Smart Card**.
4. È la Smart Card (e solo lei) a possedere la chiave a lungo termine necessaria per decifrare il messaggio (*decrypt msg2*).
5. Una volta decifrato internamente il pacchetto, la Smart Card estrae le credenziali utili (*extract credentials*)—ovvero la nuova chiave di sessione appena generata—e le fornisce alla workstation.

In questo modo, la workstation ottiene la chiave di sessione temporanea necessaria per proseguire le comunicazioni, ma la chiave a lungo termine dell'utente è rimasta al sicuro, inviolata all'interno del chip della Smart Card.

È importante sottolineare l'enorme salto generazionale e di sicurezza rispetto alla precedente tecnologia delle **carte magnetiche**. Queste ultime erano intrinsecamente insicure, paragonabili a un "libro aperto": non offrivano alcuna garanzia di riservatezza e chiunque fosse dotato di un semplice lettore poteva leggerne o clonarne il contenuto senza alcuno sforzo.

Le Smart Card (come ad esempio le *Java Card*), al contrario, essendo dei dispositivi programmabili, espongono verso l'esterno solo un'interfaccia logica rigorosa. Se il software al loro interno è progettato in maniera sicura, questa interfaccia non includerà mai un comando per estrarre la chiave

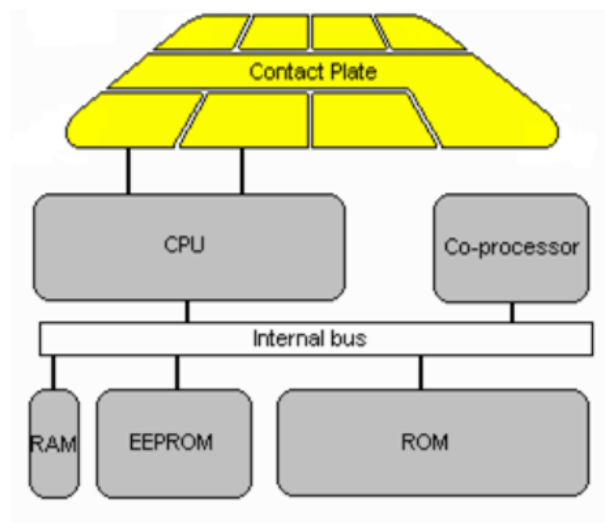
privata. Di conseguenza, l'unica opzione rimasta a un ipotetico attaccante per forzare una Smart Card si riduce a un complesso, costoso e distruttivo **attacco fisico** all'hardware del chip stesso.

Capitolo 2

Smart Card

La Smart Card è, a tutti gli effetti, un microcomputer dotato di un proprio sistema operativo, seppur in una versione semplificata. Questo tipo di dispositivi nasce principalmente per garantire la protezione dei dati (come nel caso dei bancomat) o per memorizzare in modo sicuro password e chiavi crittografiche, come abbiamo visto precedentemente con Kerberos. Le loro applicazioni sono vastissime e spaziano dalla classica telefonia mobile (le SIM), alla pay TV, fino ai sistemi di autenticazione basati sul possesso.

Rispetto al suo antenato, la carta magnetica, il salto tecnologico è enorme. Una carta magnetica, infatti, si comporta come un "libro aperto": chiunque sia dotato di un apposito lettore può leggerne (e clonarne) facilmente il contenuto. Al contrario, la Smart Card possiede un vero e proprio sistema operativo che, se ben progettato, gestisce le interfacce di comunicazione e rende difficile, se non impossibile, il dump totale della memoria in condizioni normali.



La tipica struttura hardware di una Smart Card è composta da:

- **Contact plate:** i contatti metallici esterni utilizzati per effettuare le operazioni di I/O (Input/Output).
- **Processore (CPU):** per eseguire le operazioni di calcolo e crittografia basilari.
- **ROM:** una piccola memoria a sola lettura che contiene il sistema operativo di base.
- **EEPROM:** funge da memoria semi-permanente, agendo di fatto come il disco fisso della carta. Nei vecchi telefoni cellulari, ad esempio, la rubrica e gli SMS venivano salvati esattamente qui.
- **RAM:** memoria volatile che ha la stessa identica funzione di quella presente in un classico computer.

Per quanto riguarda il Sistema Operativo, storicamente si trattava di software molto semplici e tendenzialmente monofunzione. Tuttavia, con la continua evoluzione di questi dispositivi, si è arrivati ad avere dei veri e propri OS nel senso moderno del termine. Grazie alle **Javacard**, ad esempio, è possibile eseguire bytecode direttamente sul chip. Un aneddoto molto interessante (nato dal celebre meme dell'informatica "*Can it run Doom?*") è un progetto recente in cui una smart card è stata riprogrammata per rendere possibile l'esecuzione del gioco Doom (1993), dimostrando quanto questi "mini-computer" siano diventati potenti.

2.1 Standard ISO 7816 e Attacchi

Lo **Standard ISO 7816** è la normativa internazionale che definisce le proprietà fisiche e logiche richieste a una smart card. Tra i requisiti fisici previsti ci sono, ad esempio, l'immunità ai raggi UV e l'impenetrabilità del chip (che dovrebbe essere progettato per distruggersi piuttosto che lasciarsi ispezionare fisicamente).

La realtà dei fatti, però, è che la protezione fisica assoluta non può essere garantita. Nonostante gli standard, i malintenzionati riescono a mettere a punto attacchi veri e propri per aggirare le difese, riuscendo in alcuni casi a effettuare il dump dell'intera EEPROM, a clonare la carta o ad alterare i dati sensibili in essa contenuti.

2.1.1 Tipologie di Attacco

Possiamo dividere in generale gli attacchi alle smart card in due grandi macro-categorie:

- **Invasivi:** richiedono di andare a manipolare fisicamente l'hardware della carta (es. rimuovendo strati del chip con acidi), finendo tendenzialmente per rovinare o distruggere l'involucro originale.
- **Non invasivi:** cercano di forzare o analizzare la carta (es. studiandone i consumi elettrici) senza agire fisicamente sulla sua struttura interna, preservandone così lo stato originale.

In generale, la tipica strategia di attacco è quella di effettuare attacchi invasivi in laboratorio per poi andare a effettuare attacchi non invasivi in serie.

Microprobing

Il **Microprobing** è un classico attacco di tipo invasivo, estremamente complesso e costoso, che richiede attrezzatura di laboratorio specializzata. Generalmente si compone dei seguenti step:

- **Depackaging:** estrazione fisica del chip di silicio dall'involucro di plastica della carta.
- **Riambientazione:** pulizia e preparazione del chip (spesso usando acidi per rimuovere gli strati protettivi superiori).
- **Ricostruzione del disegno (Reverse Engineering):** analisi al microscopio per mappare i collegamenti e i bus interni.

- **Alterazioni manuali:** posizionamento di micro-sonde (probe) direttamente sui circuiti del chip.
- **Lettura della memoria:** intercettazione dei segnali elettrici per estrarre i dati.

Un attacco di questo genere è mirato a fare il *dump* di informazioni sensibili che normalmente sarebbero inaccessibili. Ad esempio, la chiave a lungo termine salvata nella EEPROM, prima o poi, deve essere inviata alla CPU per eseguire le operazioni crittografiche. In quel preciso istante, la chiave viaggia in chiaro sui bus interni della carta e le micro-sonde possono intercettarla. Un principio simile (seppur non a livello di microprobing, ma di terminale) vale per la lettura del PIN: quando inseriamo il codice per un pagamento, questo passa in chiaro per un brevissimo tratto prima di essere cifrato. È in questa fase che agiscono i dispositivi malevoli (come gli *skimmer*), motivo per cui è sempre sconsigliato l'inserimento della carta in macchinette o terminali non ufficiali o manomessi.

Attacchi Software

Un classico esempio di attacco software non invasivo è quello storicamente perpetrato ai danni delle **Pay-TV**. Questi sistemi sfruttano una smart card per passare una chiave crittografica al decoder, il quale può così decodificare il segnale video. Nel momento in cui un cliente smette di pagare l'abbonamento, il gestore invia un messaggio di annullamento al decoder per disabilitare la smart card. L'attacco, in questo caso, non forza direttamente la crittografia della carta, ma consiste nel modificare il software del decoder affinché filtri o ignori del tutto questi messaggi di annullamento, permettendo alla smart card di continuare a decodificare il segnale a tempo indeterminato.

Eavesdropping (Side-Channel) e Fault Induction

Queste tecniche si basano sull'osservazione e sulla manipolazione del comportamento fisico della carta durante il suo normale funzionamento:

- **Eavesdropping (Analisi dei consumi):** Monitorando minuziosamente il consumo di corrente sulle linee di alimentazione della smart card, un attaccante può riconoscere i pattern di assorbimento energetico. Poiché istruzioni diverse (es. moltiplicazione vs. addizione) consumano quantità di corrente diverse, è possibile risalire alla chiave crittografica utilizzata durante i calcoli.

- **Fault Induction (Iniezione di errori):** In questo caso, l'attaccante va a indurre volontariamente un *flip di bit* o un malfunzionamento (glitch) per ottenere effetti specifici. Ad esempio, inviando impulsi elettromagnetici precisi o abbassando repentinamente il voltaggio (glitch di corrente), si forza la carta a comportarsi in modo anomalo. I risultati vengono poi studiati tramite complessi esperimenti di analisi differenziale.

Un esempio storico e celebre di *Fault Induction* è il glitch di corrente che esponeva la memoria dei microcontrollori **PIC16C84**. Di seguito possiamo vederne la logica vulnerabile:

```
1  b = answer_address
2  a = answer_length
3  if (a == 0) goto 8
4  transmit(*b)
5  b = b + 1
6  a = a - 1
7  goto 3
8  ...
```

Figura 2.1: Ciclo di trasmissione vulnerabile a glitch

Come possiamo intuire analizzando il codice, il chip utilizza una variabile *a* come contatore per la lunghezza del messaggio da trasmettere. Se un attaccante riesce a indurre un *glitch* di corrente nel momento esatto in cui viene eseguita la riga 6 ($a = a - 1$) o il controllo alla riga 3 ($\text{if } (a == 0)$), la condizione di uscita dal ciclo fallisce. Di conseguenza, il ciclo continua all'infinito o va in underflow, facendo sì che il comando `transmit(*b)` alla riga 4 esegua il dump non solo della risposta prevista, ma dell'intera memoria del dispositivo.

/

Capitolo 3

Crittografia Visuale

L'idea della **Crittografia Visuale** nasce nel 1994 da un'intuizione dei crittografi Naor e Shamir. L'obiettivo di questo sistema è prendere un'immagine segreta e dividerla in due immagini apparentemente casuali, chiamate **share** (condivisioni), che fungono l'una da chiave e l'altra da crittogramma.

La particolarità di questo approccio è che, preso singolarmente, uno share non rivela assolutamente alcuna informazione a un potenziale attaccante. L'unico modo per decifrare e rivelare l'immagine originale è andare a sovrapporre fisicamente (o digitalmente) la chiave al crittogramma.

Un classico esempio è il seguente:



Figura 3.1: Esempio di sovrapposizione di due share.

Come possiamo notare, se spostiamo uno dei due blocchi sopra l'altro, appare in chiaro la scritta "VCK". In pratica, in questo tipo di crittografia non è richiesta alcuna potenza di calcolo: **è il nostro occhio a fungere da decoder**.

Applicazioni e Funzionamento Logico

Questo sistema trova molte applicazioni interessanti, specialmente nel contesto bancario. Una banca, ad esempio, può spedire fisicamente lo share "chiave" (stampato su un foglio lucido trasparente) al cliente e pubblicare lo share "crittogramma" sulla propria pagina web. In questo modo, solo il cliente legittimo potrà sovrapporre il suo lucido allo schermo per leggere un

PIN o una password monouso, rendendo le intercettazioni di rete del tutto inutili.

L'Implementazione tramite Matrici 2×2

A livello logico, il sistema utilizza uno schema che riprende i concetti del **One-Time Pad** (il cifrario perfetto) e sfrutta le proprietà dell'operazione **XOR** (OR esclusivo). Si predilige l'uso logico dello XOR rispetto al classico OR perché quest'ultimo comporta una **perdita di informazione**, mentre lo XOR ci garantisce la perfetta decodifica del messaggio. Bisogna sempre ricordare che lo XOR è una delle codifiche più robuste a patto che la chiave abbia la stessa lunghezza del messaggio e che questa venga utilizzata una sola volta. Se fosse riutilizzata, allora avremmo che si potrebbe estrarre la chiave e, dunque, decodificare ogni messaggio.

Per implementare questa logica graficamente, ogni singolo pixel dell'immagine originale viene espanso in due matrici di dimensione 2×2 (una per il primo share e una per il secondo share). Ognuna di queste sottomatrici è composta da esattamente 2 pixel neri e 2 pixel bianchi (o trasparenti).

□ Affinchè l'occhio decodifichi un pixel nero, le due matrici devono essere **complementari**



□ Per un pixel bianco, devono essere **uguali**

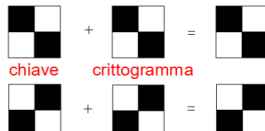


Figura 3.2: Espansione di un singolo pixel in sottomatrici 2×2 .

Il trucco risiede nella loro **sovrapposizione** (overlay):

- Se i due share hanno i pixel neri disposti in posizioni *diverse* (complementari), la loro sovrapposizione coprirà tutti e quattro gli spazi della matrice 2×2 , restituendo un macro-pixel **completamente nero**.
- Se i due share hanno i pixel neri nella *stessa* posizione, la sovrapposizione lascerà scoperti i due spazi bianchi. Il risultato sarà un macro-pixel per metà nero e per metà bianco.

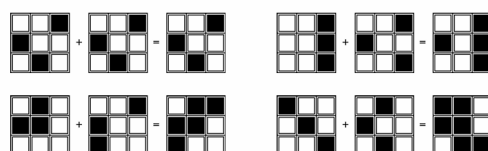
xor	0	1	overlay		
0	0	1			
1	1	0			

Figura 3.3: L'effetto visivo della sovrapposizione (XOR visivo).

Nonostante un pixel per metà nero e per metà bianco non sia tecnicamente "bianco puro", l'occhio umano interpreterà questo contrasto come una sfumatura di grigio. Questo stacco netto tra il nero totale e il grigio chiaro permette al nostro cervello di distinguere chiaramente la forma dell'immagine nascosta, facendo funzionare l'overlay grafico esattamente come una decodifica XOR.

In genere, dunque, quello che cerchiamo è un contrasto sufficientemente alto da far sì che l'occhio riesca a distinguere chiaramente la "scritta" dallo "sfondo". Sebbene sia teoricamente possibile rappresentare non solo bianco e nero puro, ma anche 3 scale di grigio intermedie con una semplice matrice 2×2 , questo abbasserebbe drasticamente il contrasto visivo, rendendo il messaggio finale molto difficile da decifrare a occhio nudo.

Per questo motivo, nel caso in cui volessimo rappresentare 4 scale di grigio ben distinte mantenendo una buona leggibilità, avremmo bisogno di espandere i pixel su una matrice 3×3 , gestendola nel seguente modo:



Anche in questo caso, la regola generale è che all'aumentare dei pixel "accesi" possiamo rappresentare più sfumature, ma paghiamo il prezzo di un contrasto sempre minore, che rende più faticosa la decodifica visiva dello share.

Un'altra nota teorica fondamentale riguarda la **perfetta sicurezza** del sistema. Proprio come in One-Time Pad, affinché lo schema sia inattaccabile, ogni share crittografico generato deve essere perfettamente equiprobabile. Cosicché, analizzando un singolo share isolato, un attaccante non possa dedurre alcuna informazione statistica sull'immagine originale sfruttando attacchi probabilistici.

L'algoritmo di codifica generico si può quindi riassumere in due fasi:

1. Per ogni pixel dell'immagine segreta, si sceglie **a caso** la matrice da associargli per il primo share (la chiave). Essendo la scelta puramente casuale, questo primo share apparirà visivamente come puro rumore bianco.
2. Successivamente, si costruisce il secondo share (il crittogramma) guardando il pixel corrispondente nell'immagine di partenza. Se il pixel originale è nero, per il secondo share si sceglie la matrice **complementare** a quella usata nella chiave; se il pixel originale è bianco, si sceglie la **stessa identica** matrice della chiave.

Così facendo, la sovrapposizione fisica dei due share darà come risultato nero totale nei punti in cui le matrici sono complementari (pixel originali neri) e un grigio composto per metà da spazi bianchi nei punti in cui le matrici sono identiche (pixel originali bianchi), riproducendo l'immagine corretta.

Esistono, inoltre, altri schemi di cifratura visuale come, ad esempio gli **schemi a soglia** in cui è necessario avere un certo numero di share totali per ottenere l'immagine originale. Allo stesso modo, esistono degli **schemi a gruppi di accesso** in cui è possibile decifrare una certa immagine solo se si possiedono contemporaneamente degli specifici gruppi di share.

Capitolo 4

Non-Ripudio

La proprietà di Non Ripudio è quella proprietà che garantisce l'**impossibilità** per un'entità di una dichiarazione o transazione di **negarne la paternità** e la validità.

Questa proprietà oltre che essere fondamentale nel mondo reale, lo è allo stesso modo per il mondo elettronico. Basti pensare a tutte le volte in cui è stato necessario fare un firma in un contratto che, quindi, garantiva la nostra adesione. Allo stesso modo, utilizzeremo la **firma elettronica** per cercare di garantire questa proprietà.

Quando andiamo a garantire questa proprietà, vorremmo farlo in un certo senso in maniera **equa** così da far sì che non solo A abbia una garanzia nei confronti di B , ma anche che B abbia una garanzia nei confronti di A . A questo punto, dunque, non siamo più in un contesto in cui A si fida di B e viceversa, ci troviamo nelle condizioni in cui ognuno dei due potrebbe fregare l'altro. Passiamo, quindi, a un modello di attaccante che abbiamo già conosciuto sotto il nome di **General Attacker** in cui ognuna delle entità partecipanti al protocollo può essere un attaccante con obiettivi diversi e non necessariamente collusi tra loro.

Nello specifico, ci concentreremo su 3 problematiche nel mondo elettronico che sono:

- Il meccanismo di **Compravendita**;
- Il sistema di **Delega**;
- Il sistema di **E-mail**.

Questi sono ambiti fondamentali nel mondo elettronico che riscontrano problemi molto simili ma in contesti diversi.

Fattori che si riveleranno importanti durante queste analisi sono principalmente l'**affidabilità del mezzo** e l'**onestà dei partecipanti**.

4.1 Problema della equa Compravendita

In un contesto reale, quello che accade è che io pago il commerciante e lui allo stesso momento mi fornisce un servizio o un bene e, seppur questo potrebbe comunque accadere, nessuno si sognerebbe mai di negare di aver appena ricevuto il denaro o il bene poiché ci sono le forze dell'ordine, tribunale e questo genere di cose che intervengono nel caso di situazioni di questo tipo.

Il problema avviene dunque nel momento in cui non si è più di presenza ma online in cui ognuno può dire di avere già inviato il bene o il denaro e, dunque, senza un qualche tipo di garanzia che questo sia realmente accaduto ci si ritrova in una fase di stallo in cui nessuno può provare che l'altro abbia in un qualche modo barato e, da qui, la necessità di volere una proprietà di Non ripudio che sia equa per ambe le parti.

Cominciamo ad analizzare i protocolli supponendo di avere:

- Due generici agenti A e B.
- Un sistema PKI che quindi fa uso di **crittografia asimmetrica**.
- Un messaggio cruciale **m**.
- E, eventualmente, un **Trusted Third Party**.

Quello che vogliamo garantire è quindi un'equa compravendita, ossia che: *A spedisca a B un messaggio m in modo tale che A ottenga evidenza che B ha ricevuto m se e solo se B ottenga evidenza che A lo abbia spedito.*

Andiamo ad analizzare i vari casi al variare dell'onestà degli agenti coinvolti e all'affidabilità del mezzo.

4.1.1 Mezzo affidabile, Agenti Onesti

Il primo caso è quello più ovvio in cui sia il mezzo è affidabile sia gli agenti sono onesti. In questo caso, banalmente, il protocollo sarà questo:

$$\begin{aligned} 1. & A \rightarrow B : f_{nro}, B, m, \text{Sign}_A(f_{nro}, B, m) \\ 2. & B \rightarrow A : f_{nrr}, A, \text{Sign}_B(f_{nrr}, A, m) \end{aligned}$$

Rispettivamente abbiamo che f_{nro} è la **Flag of non repudiation from origin** e la f_{nrr} è la **Flag of non repudiation from receiver**. La proprietà in questo caso è rispettata poiché essendo che il mezzo è affidabile e gli agenti non sono intenzionati a mentire, questi riceveranno sempre le rispettive conferme.

4.1.2 Mezzo affidabile, Agenti disonesti

Questo è già un caso più complesso. Abbiamo la certezza che i messaggi arrivano, tuttavia vogliamo impedire agli agenti di commettere azioni disoneste. Per questo caso abbiamo dunque bisogno di una sorta di *giudice imparziale* che sarà il nostro **Trusted Third Party**:

1. $A \rightarrow TTP : f_{nro}, TTP, B, m, \text{Sign}_A(f_{nro}, TTP, B, m)$
2. $TTP \rightarrow B : f_{nrs}, A, B, m, \text{Sign}_{TTP}(f_{nrs}, A, B, m)$
3. $TTP \rightarrow A : f_{nrd}, A, B, \text{Sign}_{TTP}(f_{nrd}, A, B, m)$

Vediamo dunque che quando A vuole mandare un messaggio per avere non ripudio, prima lo invia al TTP e sarà quest'ultimo a garantire che B riceva il messaggio e, quindi, fornirgli l'evidenza che A abbia inviato il messaggio e, poi, fornirà ad A stesso la conferma che B abbia ricevuto il messaggio. La proprietà è, dunque, garantita dal TTP che agendo in maniera sempre onesta e avendo che il mezzo è sempre affidabile, si fa carico del compito di inviare le varie ricevute. Abbiamo qui due notazioni diverse che sono f_{nrs} che sarebbe il flag di **Flag of non repudiation of submission** e f_{nrd} che sta per **Flag of non repudiation of delivery**. Al primo messaggio viene fornito anche un f_{nro} , questo viene fatto così che il TTP abbia la certezza che A volesse mandare quel messaggio con l'intenzione di garantire non ripudio.

4.1.3 Mezzo inaffidabile, Agenti onesti

In questo caso, abbiamo che gli agenti siano interessati a mandare sempre la loro parte di evidenza di invio e di evidenza di ricezione con l'unico problema in mezzo rappresentato dal mezzo che non è affidabile. Dunque si procede come segue:

1. $A \rightarrow B : f_{nro}, B, m, \text{Sign}_A(f_{nro}, B, m)$
2. $B \rightarrow A : f_{nrr}, A, m, \text{Sign}_B(f_{nrr}, A, m)$
3. $A \rightarrow B : f_{ack}, B, \text{Sign}_A(f_{ack}, B, m)$

A questo punto, analizziamo i punti di possibile fallimento. Nel caso in cui il primo messaggio non venga recepito da B, la proprietà è rispettata perché nessuno dei due riceve evidenza di niente. Nel caso, invece, A non ricevesse il messaggio 2, B, poiché onesto, continuerà a mandare nuovamente il messaggio 2 fino a quando non riceve l'ack. Infine, se anche B non ricevesse il messaggio 3, entrambi avrebbero comunque evidenza di invio/ricezione dunque la proprietà non sarebbe violata.¹

¹anche se B comunque continuerebbe a inviare il messaggio 2 e quindi A riinvierebbe il messaggio 3

4.1.4 Mezzo inaffidabile, Agenti disonesti

Siamo adesso arrivati al caso reale in cui tutto potrebbe andare storto. Abbiamo visto che al mezzo inaffidabile, la soluzione è la **ritrasmissione** mentre per quanto riguarda gli agenti disonesti, la soluzione è il **TTP**. Vediamo però un altro tentativo proposto nel lavoro di **Zhou-Gollman**. Questi sfruttano un'idea battezzata dal professore come **Rilascio Posticipato**:

1. Inizialmente si fornisce il crittotesto al ricevente.
2. Se il ricevente è interessato a decodificare il crittotesto, deve continuare la sessione di fatto, lasciando delle tracce.
3. Una volta ottenute queste tracce, A invia la chiave per decifare il messaggio.

Vediamo come Zhou-Gollman hanno provato a implementare questa strategia:

1. $A \rightarrow B : f_{poe}, B, c, \text{Sign}_A(f_{poe}, B, c)$
2. $B \rightarrow A : f_{acp}, A, \text{Sign}_B(f_{acp}, A, c)$
3. $A \rightarrow B : f_{nro}, B, k, \text{Sign}_A(f_{nro}, B, k)$
4. $B \rightarrow A : f_{nrr}, A, \text{Sign}_B(f_{nrr}, A, k)$

Abbiamo qui che f_{poe} è la **Flag promise of exchange** e f_{acp} è la **Flag Acknowledgement of Content Provision** che conferma l'arrivo della promessa in questo caso. Questa strategia, però, non funziona, infatti B potrebbe non inviare mai l'ultimo messaggio e aver comunque ricevuto da parte di A la f_{nro} . In realtà, pensandoci bene, questa era una strategia che, di base, non aveva alcun senso poiché, come abbiamo potuto vedere, abbiamo bisogno di un TTP per gestire l'onestà degli agenti e della ritrasmissione per far sì che ci sia affidabilità nel mezzo.

4.1.5 Protocollo Zhou-Gollman

Verifichiamo adesso il protocollo proposto da Zhou-Gollman per garantire equa compravendita. Questo sfrutta la strategia vista precedentemente di rilascio posticipato e fa uso di TTP. Una cosa di cui si deve, però, dare merito è la sostituzione del meccanismo di ritrasmissione con l'uso di FTP. In pratica, andiamo a dare a coloro che vogliono ricevere le evidenze, la responsabilità di scaricarsele dal TTP. Questo perché, sebbene il mezzo sia inaffidabile, non lo è sempre e, dunque, sarà compito degli agenti preoccuparsi

di prenderlo. Per il protocollo Zhou-Gollman saranno utilizzate le seguenti abbreviazioni:

$$\begin{aligned}
c &= m_k \\
NRO &= \text{Sign}_A(f_{nro}, B, L, c) \\
NRR &= \text{Sign}_B(f_{nrr}, A, L, c) \\
sub_k &= \text{Sign}_A(f_{sub}, B, L, k) \\
con_k &= \text{Sign}_{TTP}(f_{con}, A, B, L, k)
\end{aligned}$$

Il protocollo Zhou-Gollman è dunque:

$$\begin{aligned}
1. & A \rightarrow B : f_{nro}, B, L, c, NRO \\
2. & B \rightarrow A : f_{nrr}, A, L, NRR \\
3. & A \rightarrow TTP : f_{sub}, B, L, k, sub_k \\
4. & B \xleftarrow{FTP} TTP : f_{con}, A, B, L, k, con_k \\
5. & A \xleftarrow{FTP} TTP : f_{con}, A, B, L, k, con_k
\end{aligned}$$

Abbiamo dunque da 1 a 2, il rilascio posticipato per 3 in cui A invia la chiave k per leggere il messaggio al TTP che la renderà disponibile insieme al f_{con} che sarà è la **Flag of confirmation** in cui il TTP conferma che il suo lavoro di esporre la chiave è stato effettuato e, dunque, gli agenti sono tenuti a ritirare la chiave k . Nessuno tra A e B ci conclude qualcosa a fermarsi ai primi passi:

- Se A si ferma al passo 2, la proprietà non è violata poiché per dimostrare in tribunale che B abbia ricevuto il messaggio serve anche la conferma da parte del TTP e, quindi, senza di questa A non ha realmente nulla in mano.
- Se B si ferma al passo 1, anche in questo caso senza la conferma del TTP questo non può provare nulla e dunque la proprietà non è violata.

In sostanza, fino al passo 3 anche se imbrogliassero la proprietà non sarebbe violata e, dopo il passo 3, nessuno può più barare perché per via della disponibilità della chiave k presso il TTP. Un'altra cosa importante da sottolineare è la presenza della label L . Questa è una sorta di id che serve a effettuare il **binding** tra il crittotesto e la chiave. Infatti, senza di questa, A potrebbe inviare al TTP un chiave k errata che però apre un altro crittotesto c_2 e, quindi, ottenere comunque una f_{con} valida. Il TTP dovrebbe essere **stateful** per evitare attacchi di replay e dovrebbe avere modo di assicurarsi che la chiave sia veramente corretta per quel crittotesto.

4.1.6 Analisi del rilascio posticipato

Si noti che il primo testo è effettivamente cifrato e si può dunque dibattere sul perché di questa cosa. Infatti, gli autori stessi sostengono che questa non sia una scelta per garantire confidenzialità. Dunque, risulta essere valida anche un protocollo di questo tipo:

$$\begin{aligned} 1. & A \rightarrow TTP : f_{sub}, B, m \\ 2. & A \xleftarrow{FTP} TTP : f_{con}, m, A, B, Sign_{TTP}(A, B, m) \\ 3. & B \xleftarrow{FTP} TTP : f_{con}, m, A, B, Sign_{TTP}(A, B, m) \end{aligned}$$

Evitando così il rilascio posticipato che complica il protocollo e alleggerendo dunque tutta l'infrastruttura del protocollo.

4.2 Non ripudio in e-mail

Al giorno d'oggi ormai, l'utilizzo di e-mail è pressoché dominante qualora si voglia dialogare con qualsivoglia azienda o simili. Questo, fa sì che possa essere utile la proprietà di non ripudio nel caso in cui si voglia andare a risolvere determinati scontri legali. In Italia esiste il sistema di **PEC** cioè Posta Elettronica Certificata. Questo sistema è unico nel suo genere e garantisce non ripudio da parte di coloro che ne fanno uso e, di fatto, diventando molto usata in ambiti legali. Sebbene questo sembri risolvere il nostro problema, ci possiamo fin da subito rendere conto di una cosa: la PEC non pone alcuna nostra firma da nessuna parte e, allo stesso modo, non viene inserita alcuna firma da parte del ricevente della e-mail. Quello che fa in realtà la PEC non è altro che inviare una ricevuta al mittente riguardo al fatto che il destinatario abbia ricevuto la e-mail e una ricevuta al destinatario riguardo al fatto che il mittente gli abbia inviato una certa e-mail e, queste ricevute, vengono firmate con dal sistema stesso che offre il servizio rendendolo, di fatto, un **protocollo di Trustness** nei confronti di colui che gestisce il servizio.

4.2.1 Equo recapito

Innanzitutto, il nostro obiettivo sarà quello di ottenere **equo recapito**. Questo potrà essere espresso in due forme:

- **Debole:** Il ricevente legga l'e-mail se e solo se il mittente riceva la ricevuta di ritorno.
- **Forte:** Il ricevente legga l'e-mail, ottenendo pure evidenza che provenga del mittente, se e solo se il mittente riceva la ricevuta di ritorno.

Analizziamo queste due forme e verifichiamo il perché queste due siano diverse e in cosa lo sono. Nella forma debole, non abbiamo alcuna forma di **autenticazione** da parte del mittente rendendo possibili scenari in cui il mittente possa inviare messaggi a nome di qualcun altro senza che la proprietà sia violata. In questo caso, dunque, l'evidenza che possiede il mittente è molto più forte poiché possiede non solo il messaggio ma anche la ricevuta che autentica il destinatario facendo sì che il mittente ottenga una proprietà di primo livello che, invece, il destinatario non ha. Questo scenario non è possibile nel caso in cui si parli di equo recapito forte poiché è necessario non solo che il destinatario riceva la e-mail ma anche una prova inconfutabile che sia stato proprio il mittente a inviarglielo.

4.2.2 Protocollo Abadi et al.

Vediamo adesso di analizzare un protocollo che cerchi di realizzare questa proprietà. Prima di farlo, però, vediamo alcune precondizioni:

- Abbiamo un mittente S , un ricevente R e una e-mail m .
- Avremo un TTP **stateless**. Proprio per questo motivo, non è affidata a lui la responsabilità di evitare replay. Vedremo che un replay non viola alcuna proprietà poiché semplicemente il receiver riceverebbe nuovamente qualcosa che già possiede.
- Non abbiamo bisogno di PKI e solo il TTP ha coppia di firma e codifica.
- Ogni agente condivide una password con il TTP.
- S e R condividono una funzione **ack** di questo tipo $ack(q) = r$ sconosciuta al TTP che serve per autenticare R con S e viceversa.
- Disponibilità di SSL.

Già solo guardando queste condizioni sorgono spontanei alcuni dubbi:

- È vero che il sistema non fa uso di PKI, tuttavia abbiamo che, in pratica, **tutti hanno un segreto con tutti** poiché ogni R ha un segreto con ogni S e tutti hanno una password con il TTP. Questo è, in pratica, lo scenario classico della crittografia simmetrica che sappiamo non essere in alcun modo scalabile e, ragion per cui, facciamo uso della crittografia asimmetrica con sistema di PKI.

- Una cosa insolita è che non abbiamo che R e S condividono una chiave segreta o simili, bensì condividono una **funzione ack**. Questa funzione, sebbene vari in base a q, è chiaramente una funzione costante non tanto per il fatto che lo sia il risultato, ma quanto più per il fatto che la legge è sempre costante e allo stesso q corrisponderà sempre lo stesso r. Ciò rende dunque in realtà questo sistema un modo più comodo per tenere una sorta di segreto condiviso tra i due che, però, vedremo dopo essere molto utile al TTP.

Guardiamo adesso alcune abbreviazioni che useremo nel protocollo:

- $c = m_k$
- $h_S = Hash(q, r, c)$
- $h_R = Hash(q', r', c')$
- $S2TTP = \{S, k, R, h_S\}_{pub_{EKTTP}}$
- $DR = \{S2TTP\}_{Sign_{TTP}}$

Anche qui abbiamo alcune cose da notare:

- Innanzitutto, notiamo che h_R prende in input i corrispettivi input di h_S ma con ' sopra. Questo è segnalato in questo modo perché vedremo dopo che potrebbe accadere che q, r, c potrebbero non arrivare correttamente. Questa analisi viene fatta apertamente in questo modo perché gli autori del protocollo, a differenza di Zhou-Gollman, specificano di voler tenere in considerazione un attaccante in perenne ascolto nella rete, cioè Dolev-Yao. Vedremo dopo cosa accade nel caso in cui anche solo uno di quei parametri è diverso da quello di origine.
- $S2TTP$ è un messaggio che sarà generato si presume da S e che potrà essere letto solo ed esclusivamente da parte del TTP poiché cifrato con la chiave pubblica del TTP.

Il **protocollo proposto da Abadi et al.** è dunque il seguente:

1. $S \rightarrow R : TTP, c, q, S2TTP$
2. $R \xrightarrow{SSL} TTP : S2TTP, pwd_R, h_R$
3. $TTP \xrightarrow{SSL} R : k, h_R$
4. $TTP \rightarrow S : DR$

Analizziamo il tutto messaggio per messaggio:

1. Innanzitutto, il sender invia ad R il messaggio c cifrato seguendo quella che è la strategia del rilascio posticipato. Invia dunque q che è una sorta di challenge a cui R dovrà rispondere correttamente secondo la funzione ack che conoscono solo tra loro. Infine, viene inviato il S2TTP cifrato con la chiave pubblica del TTP. Segnaliamo che dentro S2TTP è presente h_S
2. Il receiver, non potendo fare nulla con S2TTP, prende questo messaggio e lo invia al TTP insieme alla sua password per autenticarsi al TTP. Manda, inoltre h_R . A questo punto torna utile l'analisi precedente, se anche solo uno dei parametri della funzione hash è stato modificato dalla rete o da un attaccante o è stato un tentativo di forgery, avremo che h_R sarà diverso da h_S . Se, invece, h_R è uguale ad h_S , vuol dire che dall'altra parte c'è sicuramente S agli occhi di B poiché solo lui può conoscere r che è stato generato dalla funzione segreta $ack(q) = r$ che conoscono solo loro due.
3. Il TTP vedendo S2TTP e potendolo leggere, confronta h_R e h_S e, nel caso in cui sono diverse fa abort e non prosegue il protocollo. Altrimenti, invia al receiver la chiave k per leggere il messaggio e h_R stesso per indicargli che quella chiave serve proprio per quella specifica "sessione". Infatti, h_R ha, se vogliamo, una funzione di label che, dunque, sarà diversa ad ogni crittotesto inviato da S.
4. Infine, il TTP invierà a S la conferma DR che il messaggio sia stato recapitato correttamente.

Sebbene un forgery del primo messaggio sia tecnicamente fattibile, questo non andrebbe a buon fine poiché l'attaccante non potrebbe conoscere la funzione ack segreta tra R e l'S che vuole impersonare e, dunque, il controllo del TTP poco prima del terzo messaggio, non va a buon fine. Sottolineiamo ancora una volta che **S non fa alcuna autenticazione con il TTP** bensì R lo autentica perché sa che la funzione è condivisa solo con S e solo lui poteva generare quell'hash che tra le altre cose non viene riconosciuto direttamente da R, bensì dal TTP che fa il confronto e fornisce la conferma. Chiediamoci adesso se la proprietà è quella di equo recapito debole o forte. Abbiamo che S ottiene DR che rappresenta la ricevuta di conferma di ricezione del messaggio a R, dunque, la proprietà di equo recapito debole è rispettata. Dal canto suo, invece, R ha autenticato S tramite il controllo effettuato dal TTP sugli hash ma non ha nulla che possa dimostrare realmente che dietro quel messaggio ci sia S poiché non ha una prova tangibile fornita dal TTP e la sua parola sul fatto che quell'hash sia uguale ecc. non vale nulla. Dunque, sebbene

autenticiamo S, non abbiamo una prova della sua partecipazione effettiva alla conversazione e, dunque, **non abbiamo equo recapito forte**. Per ottenere equo recapito forte, dobbiamo far sì che il TTP fornisca al receiver una prova dell'effettiva autenticazione di S. Per farlo, ci basta modificare il protocollo come segue:

- S inserisce la sua password in S2TTP. A questo modo, solo il TTP potrà leggere la password di S e, dunque, autenticandolo. Così facendo, il tentativo di forgery fallisce ancora prima del confronto degli hash perché un attaccante non potrà avere la password di S.
- Una volta effettuata l'autenticazione di S al TTP, al messaggio 3 il TTP deve menzionare S e firmare il messaggio così da avere una prova da "tribunale".

Così, e solo, in questo modo, otteniamo equo recapito forte perché il receiver, grazie al messaggio 3 può ottenere una prova dimostrabile della partecipazione di S alla conversazione.

4.2.2.1 Risoluzione delle controversie

Analizziamo velocemente cosa potrebbe accadere:

- Se S negasse di avere inviato quella e-mail, allora R potrebbe mostrare il messaggio 3 che è menziona l'invio della chiave di S per quel messaggio. Poiché il messaggio 3 è firmato dal TTP questa è un prova valida.
- Se R negasse di aver ricevuto il messaggio, allora S potrebbe mostrare *DR* che quindi inchioderebbe R poiché *DR* è prodotto dall'autorità fidata.

4.3 Non ripudio in Delega

Analizziamo adesso l'ultimo caso tra quelli proposti. Parlando di delega, ci si riferisce a quel processo in cui si **trasferisce a qualcun altro un'autorizzazione** sul fare una certa operazione agli occhi di un determinato servizio. È bene ricordare che l'autorizzazione non è un obbligo e, dunque, la delega non costringe colui che la riceve a fare alcun ché.

Il nostro obiettivo sarà ottenere **equa delega**, sia grantee colui che riceve la delega e grantor colui che la fornisce, definiamo l'equa delega come la proprietà secondo cui:

"Il grantee riceve evidenza che il grantor gli ha dato diritto su una certa operazione Ω se e solo se il grantor riceve evidenza che il grantee ha accettato responsabilità su Ω ."

4.3.1 Protocollo Crispo

Consideriamo le seguenti precondizioni prima di vedere il protocollo:

- Avremo il Grantor G e il grantee g .
- La delega servirà per svolgere il compito Ω presso un certo end-point.
- Sia G che g avranno una coppia di chiavi di firma **certificate**.
- Oltre alle chiavi di firma certificate, faremo uso di tre coppie di chiavi asimmetriche: 1 usata da G e 2 da g .
- Non faremo alcun uso di TTP.

Anche qui, prima di analizzare il protocollo, possiamo fare un'osservazione: notiamo molte più chiavi del solito. Questo è dovuto al fatto che **ognuna di queste coppie di chiavi avrà una funzione diversa**. Vedremo che avremo, ad esempio, una coppia di chiavi per effettuare la delega, una per accettarla e una per eseguirla materialmente.

Compito	Sintassi		Algoritmo	Usata da	Certificazione
Firma	priSK	pubSK	firma	G e g	Esterna
Delega	priDK	pubDK	firma	G	Interna
Accettazione	priAK	pubAK	firma	g	Interna
Esercizio	priEK	pubEK	Codifica asimm.	g	Interna

Queste chiavi avranno una certificazione "interna" cioè sono **firmate tramite le chiavi di firma già certificate di G e g** di fatto, estendendo la PKI esistente. Queste saranno chiavi che avranno senso in funzione del fatto che sono certificate da delle chiavi già certificate. Il protocollo si sviluppa dunque come segue:

1. $G \rightarrow g : (G, g, \Omega, \text{pubDK}_G)_{\text{Sign}_{\text{priSK}_G}}$
2. $g \rightarrow G : (g, \Omega, \text{pubAK}_g, \text{pubEK}_g)_{\text{Sign}_{\text{priSK}_g}}$
3. $G \rightarrow g : (g, G, \Omega, \text{pubDK}_G, \text{pubEK}_g, \text{pubAK}_g)_{\text{Sign}_{\text{priDK}_G}}$

Analizziamo anche qui il protocollo passo passo:

1. Al primo passaggio, G chiede a g se questo vuole ottenere la delega riguardo al compito Ω da parte di G . Questo primo messaggio è firmato con la chiave privata di firma di G .

2. Dopodiché g invia a G la conferma di accettazione della delega mostrando la chiave pubblica di accettazione della delega e mostrando la chiave pubblica con la quale intende esercitare il diritto di delega.
3. G a questo punto conferma il tutto firmando la delega con la sua chiave privata di delega.

Completato il protocollo, qualora g volesse esercitare il suo diritto di delega, dovrà fornire all'end-point il **token di delega** che non è altro che il messaggio 3 del protocollo, firmato con la chiave privata di accettazione di g :

$$\{(g, G, \Omega, \text{pub}DK_G, \text{pub}EK_g, \text{pub}AK_g)_{\text{Sign}_{\text{pri}DK_G}}\}_{\text{Sign}_{\text{pri}AK_g}}$$

L'end-point fungerà dunque da verificatore passivo, dovrà solo verificare che le firme nel token corrispondano alle chiavi scambiate. Verifichiamo la possibilità di ottenere dei vantaggi da parte di uno o dell'altro:

- Se G volesse ottenere un vantaggio su g , potrebbe terminare la sessione prima del messaggio 3. Tuttavia, non avrebbe modo di dimostrare che g abbia mai esercitato la delega poiché, per farlo, ha la necessità del token di delega che può generare solo g tramite le chiavi private di accettazione.
- Se g volesse ottenere un vantaggio potrebbe pensare di non andare avanti con il messaggio due, tuttavia anche in questo caso non ne otterrebbe nulla poiché non potrebbe esercitare la delega.

4.3.1.1 Risoluzione delle controversie

Vediamo cosa accade in caso di controversie:

- Supponiamo G voglia negare di aver delegato g su Ω . In tal caso g deve solamente mostrare il token di delega che, in quanto firmato con la chiave privata di delega di G , è effettivamente valida.
- Se g nega di aver accettato da G la responsabilità su Ω , allora a G basta esibire il messaggio 2. Questo, per quanto possa sembrare un vantaggio, non lo è in realtà nel caso in cui manchi una prova che g abbia esercitato Ω .
- Se g nega di aver esercitato Ω per conto di G , allora G insieme all'end-point mostra il messaggio 2 e il token di delega richiesto dall'end-point.

4.3.2 Semplificazione del Protocollo

Dal punto di vista della funzionalità, il protocollo in realtà funziona benissimo anche con le sole due chiavi di G e g . Questo perché potremmo scrivere il protocollo come segue:

1. $G \rightarrow g : \text{Sign}_{\text{pri}SK_G}(G, g, \Omega)$
2. $g \rightarrow G : \text{Sign}_{\text{pri}SK_g}(g, \Omega)$
3. $G \rightarrow g : \text{Sign}_{\text{pri}SK_G}(G, g, \Omega, Ev_msg2)$

Infatti con il primo messaggio G chiede a g se vuole la delega, con il secondo g "accetta" la delega mentre con il terzo G dice che da quel momento la delega è attiva. Infatti, verrà prodotto il token che viene presentato all'end-point come segue:

$$\{\text{Sign}_{\text{pri}SK_G}(msg3)\}_{\text{pri}SK_g}$$

L'uso di chiavi dedicate (DK , AK , EK) serve a soddisfare principi di sicurezza che vanno oltre il semplice non-ripudio:

- **Separazione dei Privilegi (Key Separation):** Usare la chiave primaria (SK) per ogni singola operazione è rischioso. Se la chiave usata per firmare i token di delega venisse compromessa, in un sistema a chiavi separate avresti compromesso solo la capacità di delegare (DK). Nella versione semplificata, avresti compromesso l'intera identità digitale dell'utente (SK).
- **Limitazione del Rischio (Proxy Keys):** Spesso il Grantee (g) deve usare la delega su macchine meno sicure o tramite software automatizzati. Usando una Execution Key (EK_g) creata ad hoc, g non deve mai "caricare" la sua chiave primaria (molto preziosa) sul server che esegue il compito Ω .
- **Prova di Intento Specifico:** Usando una Acceptance Key (AK_g), il Grantee genera una prova che non è solo una firma generica, ma un atto crittografico che significa esclusivamente "Accetto questa specifica delega". Questo previene attacchi di tipo Cross-Protocol Replay, dove una firma fatta per uno scopo viene riutilizzata per un altro.
- **Revocabilità Granulare:** È molto più facile revocare una singola chiave di delega (DK_G) o di esecuzione (EK_g) senza dover revocare il certificato principale dell'utente, cosa che richiederebbe la pubblicazione in una CRL (Certificate Revocation List) e avrebbe un impatto enorme.

Capitolo 5

Analisi Formale

Fino ad ora abbiamo analizzato i protocolli in maniera del tutto **informale**, utilizzando il linguaggio comune per ragionare sulla sicurezza di un protocollo. Chiaramente, l'analisi informale non basta per dimostrare che un protocollo è sicuro, poiché è soggetta a bias e dipende fortemente dall'esperienza di chi effettua l'analisi. Tuttavia, ci rendiamo conto che in realtà anche l'analisi informale ci può fornire qualche risultato. Di fatto, se troviamo un attacco dimostriamo chiaramente che il protocollo è insicuro nei confronti di quella specifica minaccia; tuttavia, se non lo troviamo, questo non basta a dimostrare che l'attacco non ci sia.

Vorremmo una sorta di strumento che ci dia garanzie formali riguardo alla **sicurezza** di un determinato protocollo **rispetto a una determinata proprietà**. La cosa più vicina a questo concetto che già conosciamo è la dimostrazione matematica, che certifica l'effettiva correttezza di un teorema. Queste dimostrazioni si basano su degli **assiomi** che vengono presi per veri e da cui si derivano altre verità. Possiamo dunque dire che la dimostrazione matematica non è altro che un **processo logico-deduttivo** che ci porta a dedurre ulteriori affermazioni vere, a patto che gli assiomi di partenza lo siano. Queste dimostrazioni sono **universali** e, dunque, non necessitano di un'enumerazione esaustiva di tutti i casi possibili per essere considerate valide.

Il nostro caso è però del tutto diverso: la sicurezza non è uno stato universale e, inoltre, non possiamo prendere per buoni assiomi assoluti. Quello che faremo noi sarà utilizzare un **modello finito** e definire la proprietà da ricercare come la **violazione della sicurezza** (ad esempio, la compromissione di una chiave). Infatti, se su questo modello riusciamo a dimostrare la presenza di una certa proprietà (ovvero troviamo l'attacco), allora possiamo affermare con certezza che questa vulnerabilità esiste anche nella generalizzazione del sistema. Tuttavia, **non è possibile dire che se una violazione non ap-**

pare nel nostro modello finito, allora questa non esiste nemmeno nel modello generalizzato.

Per quanto riguarda l'analisi formale, esistono diversi approcci, ognuno con le proprie peculiarità matematiche e concettuali:

- **Algebre di processo (es. CSP, π -calculus):** Modellano i protocolli come sistemi di processi concorrenti. L'attenzione è posta sulla sequenza di eventi e sulle interazioni (sincronizzazioni) tra i vari agenti sui canali di comunicazione. Sono strumenti estremamente rigorosi per descrivere il parallelismo e il flusso dei dati.
- **Logiche di Autenticazione (es. BAN Logic):** Sono note come "logiche della credenza" (*belief logics*). Invece di tracciare il singolo bit del messaggio, queste logiche modellano come evolve la conoscenza e la fiducia degli agenti passo dopo passo (ad esempio, formalizzando concetti come: "Se A riceve un messaggio cifrato con la chiave condivisa con B, allora A *crede* che B lo abbia inviato"). Sono ottime per scovare assunzioni implicite ed errate sulla fiducia.
- **Linguaggi ad hoc (es. HPSL):** Sono linguaggi di specifica creati appositamente per descrivere protocolli crittografici ad alto livello. Permettono di scrivere il protocollo in una sintassi vicina a quella umana (spesso derivata dalla notazione "Alice e Bob") per poi essere tradotti automaticamente in modelli matematici digeribili dai tool di analisi.
- **Dimostrazione automatica di teoremi (Theorem Proving, es. per induzione matematica):** Utilizza assistenti alla dimostrazione (come Isabelle/HOL o Coq) per applicare regole di inferenza logica, partendo dagli assiomi del sistema. Richiede un notevole sforzo manuale da parte dell'analista per guidare il software nella dimostrazione, ma offre la garanzia di correttezza universale per un numero illimitato di sessioni e agenti.

5.1 Model Checking

Il model checking è un metodo automatico per analizzare protocolli che mira non tanto a certificare la sicurezza assoluta, bensì a trovare degli attacchi (validazione per enumerazione). In pratica, specifichiamo il sistema target come una serie di **stati**, i quali possono essere interpretati come i fotogrammi di un video che rappresentano la configurazione del sistema in un dato istante. Lo stato può includere una rappresentazione epistemologica di ciò che conosce

o può dedurre un determinato agente. Una volta generato lo spazio degli stati e fornita in input la proprietà da verificare, si procede ad un'analisi automatica tramite una macchina a stati finiti.

Più nel dettaglio, gli stati saranno generati secondo la sintassi di un certo linguaggio di modellazione, e la stessa cosa sarà fatta per la proprietà da verificare. Tuttavia, non è detto che i due linguaggi utilizzati coincidano. Spesso, infatti, risulta più comodo utilizzare linguaggi differenti (ad esempio, un linguaggio imperativo per definire le transizioni di stato e una *logica temporale* per esprimere la proprietà desiderata).

I tool di model checking prendono in input il modello del protocollo e la proprietà da violare, per poi esplorare in automatico lo spazio degli stati. Questi strumenti enumerano tutti i possibili stati raggiungibili e verificano in ciascuno di essi se la proprietà si manifesta.

La criticità principale di questo approccio è il cosiddetto problema della **state explosion** (esplosione dello spazio degli stati). Questo fenomeno si verifica quando le variabili, le sessioni parallele e le possibili azioni degli agenti generano un numero di stati che cresce in modo esponenziale (raggiungendo facilmente milioni o miliardi di combinazioni). In questi casi, il tool esaurisce la memoria o il tempo di calcolo prima di terminare l'enumerazione.

Se ci pensiamo, è anche logico che accada questo. Di fatto, analizzare un protocollo non vuol dire solamente andare a guardare i classici stati che vengono raggiunti poiché, sebbene il protocollo sia finito, in realtà ci sono molte fonti di infinitezza quando analizziamo un protocollo:

- Potrebbero esserci **infiniti messaggi**;
- I messaggi potrebbero essere **infinitamente lunghi**;
- Possono esserci praticamente **infiniti agenti** ad eseguire il protocollo;
- L'attaccante potrebbe essere **infinitamente potente**.

Tutte queste fonti di infinitezza ci fanno capire chiaramente come un modello finito non sia abbastanza potente per poter effettuare l'analisi di un protocollo.

5.2 Theorem Proving

Come già accennato, le metodologie basate sulla dimostrazione di teoremi mirano a dare una prova di correttezza universale, evitando l'enumerazione degli stati come il model checking.

In questo ambito si fa spesso uso della cosiddetta **dimostrazione per induzione**. In breve, la dimostrazione per induzione si applica a **insiemi definiti in modo induttivo**, come l'insieme dei numeri naturali N :

- $0 \in N$
- Se $n \in N$, allora $\text{succ}(n) \in N$

Sulla base di questa struttura, possiamo definire il principio di induzione matematica per dimostrare una proprietà $P(n)$:

- **Caso base:** Si dimostra che $P(0)$ è vera.
- **Passo induttivo:** Si assume che $P(n)$ sia vera (ipotesi induttiva) e si dimostra che $P(n) \implies P(\text{succ}(n))$.
- **Conclusione:** Date queste precondizioni, possiamo affermare che la proprietà è universalmente valida: $\forall n \in N, P(n) = \text{Vero}$.

Grazie al metodo induttivo, possiamo dimostrare che valgono certe proprietà in un insieme induttivo senza dover verificare per ogni singolo elemento dell'insieme.

Sfruttando questi principi, abbiamo costruito dei tool software che sono i così detti **Proof Assistant** utili a verificare dimostrazioni matematiche già esistenti e per costruirne di nuove. Infatti, utilizzeremo uno di questi tool chiamato **Isabelle** proprio per cercare di dimostrare la sicurezza di alcuni protocolli usando delle particolari modellazioni che ci permettono di modellare il protocollo come un insieme induttivo su cui applicare il principio di induzione.

Sfrutteremo la **Higher Order Logics** una logica che deriva da:

- **Logica Proposizionale:** in cui abbiamo solamente variabili e proposizioni.
- **Logica del primo ordine:** In cui abbiamo dei quantificatori che possono essere usati solo dalle variabili.

Infatti, la Higher order logics deriva proprio da queste con l'aggiunta dei quantificatori anche per i predicati e non più solo per le variabili.

Adottando questi metodi, superiamo di gran lunga il semplice model checking passando, così, da un insieme finito ad un insieme infinito permettendoci dunque di dimostrare la sicurezza di un protocollo.

5.2.1 Object Level vs Meta Level

Nell'analisi formale dei protocolli, è fondamentale distinguere tra il livello della specifica e il livello del ragionamento logico applicato ad essa.

- **Object Level:** Rappresenta il livello del linguaggio di specifica vero e proprio (nel nostro caso, *Higher Order Logic* - HOL), chiamato anche *logica dell'oggetto*. Qui vengono definiti i messaggi, gli agenti e le regole del protocollo.
- **Meta Level:** Rappresenta il livello del ragionamento astratto. È il piano in cui "ragioniamo sulla logica" per dimostrare se una proprietà è soddisfatta o meno.

Questa distinzione si riflette nell'uso di due diversi simboli per l'implicazione:

Implicazione a livello oggetto ($\rightarrow$): Utilizzata all'interno delle formule della specifica HOL (freccia singola).

Implicazione a meta-livello ($\implies$): Utilizzata per denotare una deduzione logica nel processo di dimostrazione (freccia doppia).

Esempio di ragionamento combinato Se vogliamo esprimere che, sapendo che $P(X) \rightarrow Q(X)$ è vero (livello oggetto) e che $P(X)$ è verificato, allora possiamo dedurre $Q(X)$ (ragionamento meta), scriveremo:

$$((P(X) \rightarrow Q(X)) \wedge P(X)) \implies Q(X)$$

5.3 Isabelle

Isabelle è un proof assistant che fa uso di Higher Order Logic automatizzando e semplificando di molto la costruzione e la verifica di dimostrazioni formali.

Questo framework sfrutta delle librerie dette **teorie** in cui sono presenti lemmi e definizioni già dimostrate a cui possiamo attingere per effettuare ulteriori dimostrazioni. A questo punto, la domanda rimane sul come possiamo realmente definire il protocollo in termini di insieme induttivo. Per farlo, useremo:

- **Eventi:** che sono delle azioni basilari rappresentate come un set di regole.

- **Tracce:** che rappresentano una delle possibili sequenze di eventi mediante una lista.

Vedremo che gli eventi sono definiti proprio per induzione avendo un **caso base che sarà tendenzialmente l'evento NIL** e poi una **serie di passi induttivi** che permettono la creazione di più tracce. Ogni evento potrebbe avere delle precondizioni secondo una precisa sintassi da eseguire che vedremo a breve nel dettaglio.

Le dimostrazioni vengono dunque costruite dall'utente partendo da fatti di base e costruendo via via dimostrazioni sempre più complesse. Ogni dimostrazione è controllata dal sistema che ci assicura che questa sia **logicamente corretta** nella sua costruzione. Naturalmente, l'uso di questi tool non ci garantisce che se non riusciamo a dimostrare una cosa, l'attacco esiste realmente poiché è possibile che siamo noi a non riuscire a farlo. Allo stesso modo, è possibile che modellando male il problema dimostriamo una proprietà che in realtà non è quella sperata. Dunque, è bene rendersi conto che, per quanto questi siano strumenti potenti, in realtà la cosa più potente è il tipo di modellazione che facciamo e, cioè, il nostro lavoro.

5.3.1 I Tipi Fondamentali (Datatypes)

La modellazione inizia con la definizione di tre tipi di dati principali che compongono l'universo del protocollo:

- **agent:** Definisce gli attori del sistema. Include il *Server* (TTP), gli agenti onesti (*Friend nat*) e l'attaccante (*Spy*).
- **msg:** È un tipo ricorsivo che descrive la struttura dei messaggi. Comprende nomi di agenti, numeri (timestamp), *nonce* (numeri casuali freschi), chiavi crittografiche, hash, coppie di messaggi concatenati e messaggi cifrati (*Crypt*).
- **event:** Rappresenta le azioni che possono popolare una traccia. Gli eventi base sono *Says* (invio), *Gets* (ricezione) e *Notes* (annotazione interna/memoria).

5.3.2 Operatori di Conoscenza

Per modellare un attaccante attivo secondo il modello di Dolev-Yao, Isabelle utilizza tre operatori induttivi che definiscono cosa un agente può dedurre dal traffico intercettato:

- **parts**: Estrae ricorsivamente tutti i componenti di un messaggio, inclusi i contenuti dei crittotesti come se fossero "aperti". Viene usato per definire proprietà astratte, ma non rappresenta la capacità reale dell'attaccante.
- **analz**: Rappresenta la capacità di analisi realistica. Estrae i componenti di un messaggio ma può "aprire" un crittotesto solo se la chiave necessaria è già presente nell'insieme di conoscenza.
- **synth**: Rappresenta la capacità di sintesi. Permette all'attaccante di creare nuovi messaggi concatenando elementi noti, generando hash o cifrando dati con chiavi di cui è già in possesso.
- **used**: L'operatore `used` comprende tutto ciò che è stato detto tramite l'operatore `says` e `notes`. `Used` skipa le `gets` poiché prima di queste deve esserci obbligatoriamente una `Says`.
- **knows**: Questo operatore funziona allo stesso modo di `used` ed esprime la conoscenza di un agente sulla traccia. Questo operatore distingue tra il caso `spy` e non, così da poter modellare al meglio l'attaccante. Infatti, la spia può conoscere non solo tutti i messaggi ma anche tutto ciò che conoscono coloro che sono descritti come **bad** che, in qualche modo, sono stati compromessi.¹ Tutti gli altri, invece, conoscono solo quello che inviano/ricevono.

5.3.3 Definizione Induttiva del Protocollo

Il set di regole definisce l'appartenenza di una lista di eventi all'insieme `flp` (il modello del protocollo). Ogni regola (tranne il caso base) ha la forma: "Se `evs` è una traccia valida e le precondizioni sono soddisfatte, allora l'estensione della traccia con il nuovo evento è ancora una traccia valida".

Il protocollo (nel nostro esempio denominato `flp`) è definito come l'insieme di tutte le tracce valide generate dalle seguenti regole induttive:

- **Nil**: Il caso base che stabilisce che la traccia vuota `([])` appartiene sempre al protocollo..
- **Fake**: Modella l'intruso. Se `evsf` è una traccia valida, lo *Spy* può inviare a chiunque un messaggio `X` generato tramite `synth (analz (knows Spy evsf))`.²

¹Un TTP non può mai essere `Bad` poiché, per definizione, è un'entità fidata.

²Con `knows Spy` intendiamo generalmente l'insieme di tutti i messaggi inviati.

- **FLP1:** Regola di inizializzazione. Se Na è una *nonce* fresca (non presente in $used\ evs1$), l'agente A può inviare a B il primo messaggio $\{|Agent\ A, Nonce\ Na|\}$. Colui che invia il messaggio deve essere lo stesso dichiarato all'interno.
- **FLP2:** Regola di risposta. Se B riceve (*Gets*) il messaggio di A , può rispondere inviando la *nonce* Na cifrata con la propria chiave privata ($prSK\ B$).
- **Recp:** Modella il mezzo di comunicazione inaffidabile. Se un messaggio è stato inviato (*Says*), la regola permette di estendere la traccia con l'evento di ricezione (*Gets*) da parte del destinatario.

Ecco un esempio di come vengono visti dei teoremi e come vengono visti i teoremi e come si risolvono:

```

theorem Spy-not-see-NA:
  [|Says A B (Crypt(pubEK B) {|Nonce NA, Agent A|}) ∈ set evs;
   A ∉ bad; B ∉ bad; evs ∈ ns-public|]
  ⇒ Nonce NA ∉ analz (spies evs)
apply (erule rev-mp, erule ns-public.induct)
apply simp-all
  apply (rule conjI, clarify)
  apply clarify

  apply (drule Fake-analz-insert [THEN subsetD], simp)
  apply simp
  apply (blast dest: unique-NA intro: no-nonce-NS1-NS2)+
done

```

Come notiamo in figura, per dimostrare il teorema effettuiamo una serie di passaggi logici tramite "applicazioni" di altri teoremi o lemmi precedentemente dimostrati.